than a certain amount of time. If the answer comes later, you can't be sure if it reflects the market, and it is therefore considered a failed answer.

On the other hand, you tell the wallet to verify that it has the requested money. The bet instructs the wallet to reserve funds and eventually (at most once) gets an answer. (If you chose to ask instead of tell in this situation, you would be chaining questions together, which is not advisable. Figuring out what timeout is needed for the total length of nested requests is usually a losing battle.)

Let's get to the details of the bet. Be aware that this is by far the most complicated of the three actors.

The protocol has two kinds of commands—those that expect a reply and those that don't:

```
object Bet {
  ...

  sealed trait Command
  trait ReplyCommand extends Command with CborSerializable {
    def replyTo: ActorRef[Response]
  }
  ...
}
```

The instances with `replyTo` are for external communication, and the others are for internal communication. That is why the external messages are public and the others are private, as you can see in the next two snippets.

PUBLIC PROTOCOL